

Analisis, Perancangan, dan Implementasi Basis Data Relasional pada Sistem Rekam Medis Klinik

LAPORAN PROJECT BASED LEARNING (PjBL) MATA KULIAH SISTEM BASIS DATA



Dosen Pengampu:
Ferdi Chahyadi, S.Kom., M.T.

Disusun Oleh:
Kelompok 9

M. Fachri Reza	2501020121
Mahesa Bimo Abiyyu	2501020128
Rezza Firnando	2501020090
Lukman Nur Hakim	2501020129

**Program Studi Teknik Informatika
Fakultas Teknik dan Teknologi Kemaritiman
Universitas Maritim Raja Ali Haji
2026**

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga laporan tugas Project-Based Learning (PjBL) yang berjudul “Perancangan Basis Data Sistem Rekam Medis Klinik” ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai bentuk pertanggungjawaban atas pelaksanaan tugas proyek dalam mata kuliah Sistem Basis Data. Melalui proyek ini, kami berupaya merancang sebuah sistem basis data yang terstruktur dan terstandarisasi untuk mendukung operasional rekam medis di lingkungan klinik.

Kami mengucapkan terima kasih kepada dosen pengampu mata kuliah Sistem Basis Data yang telah membimbing dan mengarahkan kami selama proses perancangan berlangsung. Tak lupa, kami juga berterima kasih kepada seluruh anggota kelompok yang telah bekerja sama dengan penuh dedikasi.

Kami menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan demi perbaikan di masa mendatang.

Tanjungpinang, 26 juni 2026
Kelompok 9

DAFTAR ISI

BAB I.....	6
PENDAHULUAN	6
1.1 Latar Belakang	6
1.2 Rumusan Masalah.....	6
1.3 Tujuan	6
1.4 Batasan Masalah	6
BAB II.....	8
LANDASAN TEORI.....	8
2.1 Basis Data	8
2.2 Entity Relationship Diagram (ERD).....	8
2.3 Normalisasi Data.....	8
2.4 SQL (Structured Query Language).....	9
BAB III	10
ANALISIS DAN PERANCANGAN SISTEM	10
3.1 Analisis Kebutuhan.....	10
3.1.1 kebutuhan data	10
3.1.2 Identifikasi Aktor	10
3.1.3 Proses Bisnis	11
3.2 ERD (Entity Relationship Diagram).....	11
3.2.1 Penjelasan entitas dan relasi.....	12
3.3 Normalisasi Data.....	12
3.3.1 Bentuk Normal Pertama (1NF).....	12
3.3.2 Bentuk Normal Kedua (2NF).....	13
3.3.3 Bentuk Normal Ketiga (3NF)	13
BAB IV	14
IMPLEMENTASI.....	14
4.1 Desain Tabel	14
4.1.1 tabel pasien.....	14
4.1.2 tabel dokter.....	14
4.1.3 tabel kunjungan.....	14
4.2 SQL DDL (Data Definition Language)	15
4.3 SQL DML (Data Manipulation Language).....	16
BAB V	17
PENGUJIAN DAN QUERY No.....	17

5.1 Skenario Pengujian	17
5.1 QUERY SQL	18
5.2 Hasil Pengujian	19
BAB VI.....	22
PENUTUP.....	22
6.1 Kesimpulan	22
6.2 Saran	22
LAMPIRAN.....	23

DAFTAR GAMBAR

Gambar 3. 1 Entity Relationship Diagram.....	11
Gambar 3. 2 (2NF).....	13
Gambar 4. 1 SQL	15
Gambar 4. 2 SQL	15
Gambar 4. 3 SQL	15
Gambar 4. 4 SQL MDL	16
Gambar 4. 5 SQL MDL	16
Gambar 5. 1 QUERY 1.....	19
Gambar 5. 2 QUERY 2.....	19
Gambar 5. 3 QUERY 3.....	20
Gambar 5. 4 QUERY 4.....	20
Gambar 5. 5 QUERY 5.....	20
Gambar 5. 6 QUERY 6.....	20
Gambar 5. 7 QUERY 7.....	20
Gambar 5. 8 QUERY 8.....	21
Gambar 5. 9 QUERY 9.....	21
Gambar 5. 10 QUERY 10.....	21
Lampiran 1- ERD Full Size	23
Lampiran 2 - File SQL Lengkap	23
Lampiran 3- Dataset Uji.....	32
Lampiran 4- Screenshot Hasil Query.....	32
Lampiran 5- Link Repository GITHUB	34

BAB I

PENDAHULUAN

1.1 Latar Belakang

Klinik saat ini masih menggunakan pencatatan rekam medis secara manual melalui dokumen kertas dan penggunaan file spreadsheet yang terpisah-pisah antara bagian pendaftaran, ruang pemeriksaan dokter, dan apotek klinik. Kondisi ini menyebabkan berbagai permasalahan operasional yang menghambat kualitas pelayanan kesehatan.

Pencatatan yang tidak terintegrasi mengakibatkan data tidak terstruktur, redundansi data pasien sering terjadi, dan riwayat penyakit pasien sulit dilacak. Selain itu, klinik juga mengalami ketidaksesuaian antara stok obat fisik dengan catatan di komputer karena tidak adanya pencatatan pengeluaran obat yang terintegrasi dengan resep dokter.

Oleh karena itu, diperlukan sebuah sistem basis data relasional yang terpusat untuk mengintegrasikan alur pelayanan klinik, mulai dari pendaftaran pasien, pencatatan pemeriksaan dan diagnosa, hingga pengeluaran obat dari apotek. Sistem ini diharapkan mampu menjamin integritas data, menghilangkan redundansi, serta memudahkan pengaksesan riwayat rekam medis pasien.

1.2 Rumusan Masalah

1. Bagaimana merancang basis data rekam medis klinik yang terstruktur dan bebas dari redundansi data menggunakan proses normalisasi?
2. Bagaimana mengintegrasikan data master (pasien, dokter, obat) dengan data transaksional (kunjungan, diagnosa, resep) dalam satu skema basis data relasional?
3. Bagaimana menangani relasi Many-to-Many antara kunjungan dan diagnosa secara tepat dalam perancangan basis data?
4. Bagaimana mengimplementasikan basis data tersebut menggunakan SQL DDL dan DML sehingga dapat menghasilkan query laporan yang dibutuhkan klinik?

1.3 Tujuan

1. Merancang basis data rekam medis klinik yang terstruktur melalui proses normalisasi hingga bentuk 3NF.
2. Mengintegrasikan seluruh data master dan data transaksional dalam satu skema relasional yang saling terhubung.
3. Mengimplementasikan rancangan basis data menggunakan SQL DDL dan mengisi data awal menggunakan SQL DML.
4. Menghasilkan minimal 10 query SQL yang dapat digunakan untuk kebutuhan operasional dan pelaporan klinik.

1.4 Batasan Masalah

- Proyek ini berfokus pada perancangan dan implementasi basis data relasional, tidak mencakup pengembangan antarmuka pengguna (frontend/UI).

- Sistem yang dirancang mencakup modul pendaftaran pasien, pencatatan kunjungan, diagnosa, dan resep obat.
- Implementasi menggunakan sistem manajemen basis data (DBMS) MySQL/MariaDB.
- Data yang digunakan bersifat fiktif dan hanya untuk keperluan pengujian dan demonstrasi.
- Fitur manajemen stok obat secara real-time tidak termasuk dalam lingkup proyek ini.

BAB II

LANDASAN TEORI

2.1 Basis Data

Basis data (database) adalah sekumpulan data yang terorganisir, saling berhubungan, dan disimpan secara sistematis dalam media penyimpanan komputer sehingga dapat diakses, dikelola, dan diperbarui dengan mudah. Sistem Manajemen Basis Data (DBMS) adalah perangkat lunak yang digunakan untuk mendefinisikan, membuat, memelihara, dan mengontrol akses ke basis data.

Pada sistem basis data relasional, data diorganisasikan ke dalam bentuk tabel (relasi) yang terdiri atas baris (tuple/record) dan kolom (atribut). Setiap tabel memiliki Primary Key (PK) yang berfungsi sebagai pengenal unik setiap baris, dan Foreign Key (FK) yang digunakan untuk membangun relasi antar tabel.

2.2 Entity Relationship Diagram (ERD)

Entity Relationship Diagram (ERD) adalah model konseptual yang menggambarkan struktur data dan hubungan antar entitas dalam suatu sistem. ERD terdiri atas tiga komponen utama:

- Entitas (Entity): Objek atau konsep yang memiliki data yang ingin disimpan. Contoh: PASIEN, DOKTER.
- Atribut (Attribute): Properti atau karakteristik yang dimiliki oleh sebuah entitas. Contoh: Nama_Pasien, Tgl_Lahir.
- Relasi (Relationship): Hubungan yang terjadi antara dua atau lebih entitas. Contoh: PASIEN “melakukan” KUNJUNGAN.

Kardinalitas relasi menggambarkan batasan jumlah keterlibatan suatu entitas dalam relasi, meliputi: One-to-One (1:1), One-to-Many (1:N), dan Many-to-Many (M:N).

2.3 Normalisasi Data

Normalisasi adalah proses penyusunan tabel dalam basis data relasional untuk mengurangi redundansi data dan meningkatkan integritas data. Proses normalisasi dilakukan secara bertahap melalui beberapa tahapan bentuk normal (Normal Form/NF):

- UNF (Unnormalized Form): Tabel awal yang belum dinormalisasi, dapat mengandung repeating group.
- 1NF (First Normal Form): Setiap atribut harus bersifat atomik dan tidak ada repeating group.
- 2NF (Second Normal Form): Memenuhi 1NF dan tidak ada Partial Dependency – semua atribut non-PK harus bergantung penuh pada keseluruhan Primary Key.
- 3NF (Third Normal Form): Memenuhi 2NF dan tidak ada Transitive Dependency – atribut non-PK tidak boleh bergantung pada atribut non-PK lainnya.

2.4 SQL (Structured Query Language)

Normalisasi adalah proses penyusunan tabel dalam basis data relasional untuk mengurangi redundansi data dan meningkatkan integritas data. Proses normalisasi dilakukan secara bertahap melalui beberapa tahapan bentuk normal (Normal Form/NF):

- UNF (Unnormalized Form): Tabel awal yang belum dinormalisasi, dapat mengandung repeating group.
- 1NF (First Normal Form): Setiap atribut harus bersifat atomik dan tidak ada repeating group.
- 2NF (Second Normal Form): Memenuhi 1NF dan tidak ada Partial Dependency – semua atribut non-PK harus bergantung penuh pada keseluruhan Primary Key.
- 3NF (Third Normal Form): Memenuhi 2NF dan tidak ada Transitive Dependency – atribut non-PK tidak boleh bergantung pada atribut non-PK lainnya.

BAB III

ANALISIS DAN PERANCANGAN SISTEM

3.1 Analisis Kebutuhan

3.1.1 kebutuhan data

Berikut adalah 11 kebutuhan fungsional yang harus dapat difasilitasi oleh struktur basis data:

No	Kebutuhan Fungsional	Entitas yang Terlibat
KF-01	Sistem menyimpan data master pasien	PASIEN
KF-02	Sistem mengelola data dokter beserta spesialisasi	DOKTER
KF-03	Sistem mencatat setiap kunjungan pasien	KUNJUNGAN, PASIEN, DOKTER
KF-04	Sistem merekam tanda vital pasien saat pemeriksaan	KUNJUNGAN
KF-05	Sistem mencatat hasil diagnosa (termasuk kode ICD-10)	DIAGNOSA, DETAIL_DIAGNOSA
KF-06	Sistem mencatat detail resep obat yang diberikan dokter	DETAIL_RESEP, OBAT
KF-07	Sistem mengelola inventori/stok obat klinik	OBAT
KF-08	Sistem mencatat riwayat pengeluaran obat berdasarkan resep	DETAIL_RESEP
KF-09	Sistem menampilkan riwayat rekam medis pasien	PASIEN, KUNJUNGAN, DETAIL_DIAGNOSA
KF-10	Sistem menghasilkan laporan statistik penyakit terbanyak	DIAGNOSA, DETAIL_DIAGNOSA
KF-11	Sistem menghasilkan laporan obat yang paling sering diresepkan	OBAT, DETAIL_RESEP

.

3.1.2 Identifikasi Aktor

Sistem ini berinteraksi dengan empat aktor utama dalam operasional klinik:

1. Admin Klinik: Bertanggung jawab atas pengelolaan data master (data pasien, data dokter, dan data master obat) serta pembuatan laporan statistik.

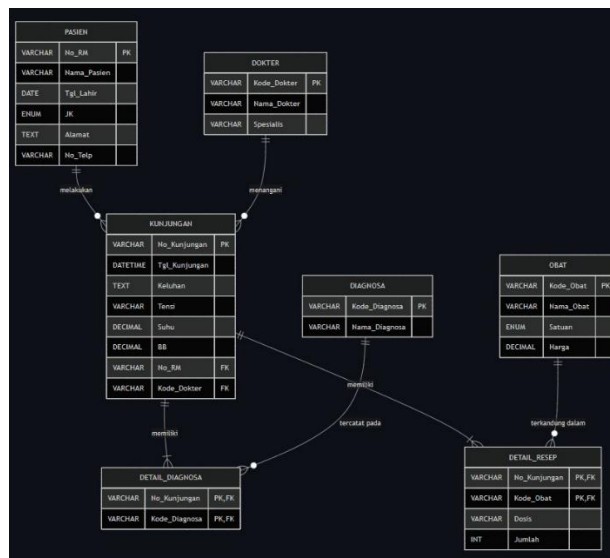
2. Perawat: Bertanggung jawab atas pendaftaran kunjungan pasien dan pencatatan tanda vital awal (tensi, suhu, berat badan).
3. Dokter: Bertanggung jawab melakukan pemeriksaan, mencatat hasil diagnosa, memberikan resep obat, dan melihat riwayat rekam medis pasien.
4. Apoteker: Bertanggung jawab memverifikasi resep dan mencatat pengeluaran obat dari inventori klinik.

3.1.3 Proses Bisnis

Proses bisnis dimulai ketika pasien datang ke klinik. Admin melakukan verifikasi data pasien jika belum terdaftar, data demografis pasien dicatat dan sistem menerbitkan Nomor Rekam Medis (No_RM) sebagai identitas unik. Pasien yang sudah terdaftar langsung melanjutkan ke tahap berikutnya. Perawat kemudian mendaftarkan kunjungan pasien dengan mencatat keluhan utama dan menentukan dokter yang akan menangani. Sistem menerbitkan Nomor Kunjungan (No_Kunjungan) sebagai acuan transaksi. Sebelum memasuki ruang dokter, perawat mengukur dan mencatat tanda-tanda vital pasien meliputi tekanan darah, suhu tubuh, dan berat badan ke dalam data kunjungan.

Dokter selanjutnya melakukan pemeriksaan dan menetapkan satu atau lebih diagnosa penyakit. Karena dalam satu kunjungan pasien dapat memiliki lebih dari satu diagnosa sekaligus, setiap diagnosa dicatat secara terpisah dengan merujuk pada kode diagnosa yang tersedia. Setelah diagnosa ditetapkan, dokter menuliskan resep yang memuat nama obat, dosis, dan jumlah untuk kemudian diproses oleh apoteker. Apoteker memverifikasi resep berdasarkan No_Kunjungan, menyiapkan obat, dan menyerahkannya kepada pasien. Seluruh data yang terbentuk dari proses ini — mulai dari data pasien, kunjungan, diagnosa, hingga resep tersimpan secara terintegrasi dalam basis data dan dapat ditelusuri kapan saja sebagai riwayat rekam medis. Data yang terakumulasi juga dimanfaatkan oleh Admin dan Dokter untuk menghasilkan laporan statistik penyakit terbanyak dan obat yang paling sering diresepkan sebagai bahan evaluasi dan pengadaan stok klinik.

3.2 ERD (Entity Relationship Diagram)



Gambar 3. 1 Entity Relationship Diagram

3.2.1 Penjelasan entitas dan relasi

Entitas

Sistem Rekam Medis Klinik dibangun dari 7 entitas yang masing-masing merepresentasikan objek data utama dalam operasional klinik. Entitas PASIEN menyimpan data identitas dan kontak seluruh pasien yang pernah berkunjung ke klinik, di mana setiap pasien diidentifikasi secara unik menggunakan Nomor Rekam Medis (No_RM). Entitas DOKTER menyimpan data identitas dokter beserta spesialisasinya yang diidentifikasi menggunakan Kode_Dokter. Entitas KUNJUNGAN merupakan entitas transaksi utama yang mencatat setiap kejadian pemeriksaan pasien di klinik meliputi tanggal kunjungan, keluhan, dan tanda-tanda vital, dengan No_Kunjungan sebagai identitas uniknya.

Selain ketiga entitas tersebut, terdapat dua entitas master yang bersifat referensi yaitu DIAGNOSA dan OBAT. DIAGNOSA menyimpan daftar penyakit beserta kodenya sehingga data diagnosa tidak perlu diulang setiap kali dicatat dalam kunjungan. OBAT menyimpan data inventori obat klinik meliputi nama, satuan, dan harga per satuan yang digunakan sebagai acuan dalam penulisan resep. Dua entitas terakhir merupakan entitas perantara yang lahir dari proses normalisasi. DETAIL_DIAGNOSA hadir sebagai solusi atas relasi Many-to-Many antara KUNJUNGAN dan DIAGNOSA, menyimpan kombinasi No_Kunjungan dan Kode_Diagnosa sehingga satu kunjungan dapat memiliki lebih dari satu diagnosa tanpa terjadi redundansi. DETAIL_RESEP menghubungkan KUNJUNGAN dengan OBAT sekaligus menyimpan atribut tambahan berupa Dosis dan Jumlah yang bersifat spesifik terhadap setiap pemberian obat dalam satu kunjungan.

Relasi

Relasi antar entitas dalam sistem ini terbentuk berdasarkan proses bisnis yang telah dianalisis. PASIEN dan KUNJUNGAN memiliki relasi One-to-Many (1:N), di mana satu pasien dapat melakukan banyak kunjungan namun setiap kunjungan hanya dimiliki oleh satu pasien. Demikian pula DOKTER dan KUNJUNGAN memiliki relasi One-to-Many (1:N), karena satu dokter dapat menangani banyak kunjungan tetapi setiap kunjungan hanya ditangani oleh satu dokter.

Adapun KUNJUNGAN dan DIAGNOSA memiliki relasi Many-to-Many (M:N) yang diwakili oleh entitas perantara DETAIL_DIAGNOSA, mengingat satu kunjungan dapat menghasilkan lebih dari satu diagnosa dan satu jenis diagnosa dapat muncul pada banyak kunjungan yang berbeda. Hal yang sama berlaku pada relasi KUNJUNGAN dan OBAT yang juga bersifat Many-to-Many (M:N) dan diwakili oleh entitas perantara DETAIL_RESEP, karena satu kunjungan dapat meresepkan lebih dari satu jenis obat dan satu jenis obat dapat diresepkan pada banyak kunjungan yang berbeda.

3.3 Normalisasi Data

3.3.1 Bentuk Normal Pertama (1NF)

Berisi seluruh atribut yang dicatat dalam formulir, termasuk repeating group untuk Diagnosa dan Obat:

```
No Kunjungan, Tgl Kunjungan, Keluhan, Tensi, Suhu, BB, No RM,
Nama_Pasien,
Tgl_Lahir, JK, Alamat, No_Telp, Kode_Dokter, Nama_Dokter, Spesialis,
{Kode_Diagnosa, Nama_Diagnosa},
```

{Kode_Obat, Nama_Obat, Satuan, Harga, Dosis, Jumlah

3.3.2 Bentuk Normal Kedua (2NF)

Ketergantungan	Atribut yang Bergantung	Tabel Hasil
Bergantung pada No_Kunjungan	Tgl_Kunjungan, Keluhan, Tensi, Suhu, BB, No_RM, Kode_Dokter	KUNJUNGAN
Bergantung pada No_RM	Nama_Pasien, Tgl_Lahir, JK, Alamat, No_Telp	PASIEN
Bergantung pada Kode_Dokter	Nama_Dokter, Spesialis	DOKTER
Bergantung pada Kode_Diagnosa	Nama_Diagnosa	DIAGNOSA
Bergantung penuh pada PK	-	DETAIL_DIAGNOSA (No_Kunjungan , Kode_Diagnosa)

Gambar 3. 2 (2NF)

3.3.3 Bentuk Normal Ketiga (3NF)

Syarat: 2NF + tidak ada Transitive Dependency (atribut non-PK bergantung pada atribut non-PK lainnya). Setelah dilakukan pengecekan pada seluruh tabel hasil 2NF:

- PASIEN: Semua atribut bergantung penuh pada No_RM. Tidak ada transitive dependency.
- DOKTER: Semua atribut bergantung penuh pada Kode_Dokter. Tidak ada transitive dependency.
- KUNJUNGAN: Semua atribut bergantung penuh pada No_Kunjungan. Tidak ada transitive dependency.
- DIAGNOSA: Semua atribut bergantung penuh pada Kode_Diagnosa. Tidak ada transitive dependency.
- OBAT: Semua atribut bergantung penuh pada Kode_Obat. Tidak ada transitive dependency.
- DETAIL_DIAGNOSA: Hanya berisi FK, tidak ada atribut non-key.
- DETAIL_RESEP: Dosis dan Jumlah bergantung penuh pada kombinasi PK.

Kesimpulan: Tidak ditemukan Transitive Dependency. Skema 2NF sudah memenuhi syarat 3NF (2NF = 3NF).

BAB IV

IMPLEMENTASI

4.1 Desain Tabel

4.1.1 tabel pasien

Nama Field	Tipe Data	Keterangan	Deskripsi
No_RM	VARCHAR(10)	PK, NOT NULL	Nomor Rekam Medis unik pasien
Nama_Pasien	VARCHAR(100)	NOT NULL	Nama lengkap pasien
Tgl_Lahir	DATE	NOT NULL	Tanggal lahir pasien
JK	ENUM('L','P')	NOT NULL	Jenis kelamin (L/P)
Alamat	TEXT	NULL	Alamat lengkap pasien
No_Telp	VARCHAR(15)	NULL	Nomor telepon pasien

4.1.2 tabel dokter

Nama Field	Tipe Data	Keterangan	Deskripsi
Kode_Dokter	VARCHAR(10)	PK, NOT NULL	Kode unik dokter
Nama_Dokter	VARCHAR(100)	NOT NULL	Nama lengkap dokter
Spesialis	VARCHAR(100)	NULL	Spesialisasi dokter

4.1.3 tabel kunjungan

Nama Field	Tipe Data	Keterangan	Deskripsi
No_Kunjungan	VARCHAR(15)	PK, NOT NULL	Nomor unik kunjungan pasien
Tgl_Kunjungan	DATETIME	NOT NULL	Tanggal dan waktu kunjungan
Keluhan	TEXT	NULL	Keluhan utama pasien
Tensi	VARCHAR(10)	NULL	Tekanan darah (misal 120/80)
Suhu	DECIMAL(4,1)	NULL	Suhu tubuh dalam Celsius
BB	DECIMAL(5,2)	NULL	Berat badan dalam kilogram
No_RM	VARCHAR(10)	FK, NOT NULL	Referensi ke tabel PASIEN
Kode_Dokter	VARCHAR(10)	FK, NOT NULL	Referensi ke tabel DOKTER

4.2 SQL DDL (Data Definition Language)

```
MariaDB [(none)]> use rekam_medis_klinik;
Database changed
MariaDB [rekam_medis_klinik]> CREATE TABLE PASIEN (
  ->   No_RM VARCHAR(10) PRIMARY KEY,
  ->   Nama_Pasien VARCHAR(100) NOT NULL,
  ->   Tgl_Lahir DATE,
  ->   JK CHAR(1),
  ->   Alamat TEXT,
  ->   No_Telp VARCHAR(15)
  -> );
Query OK, 0 rows affected (0.066 sec)

MariaDB [rekam_medis_klinik]> CREATE TABLE DOKTER (
  ->   Kode_Dokter VARCHAR(10) PRIMARY KEY,
  ->   Nama_Dokter VARCHAR(100) NOT NULL,
  ->   Spesialis VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.020 sec)

MariaDB [rekam_medis_klinik]>
MariaDB [rekam_medis_klinik]> CREATE TABLE DIAGNOSA (
  ->   Kode_Diagnosa VARCHAR(10) PRIMARY KEY,
  ->   Nama_Diagnosa VARCHAR(150) NOT NULL
  -> );
Query OK, 0 rows affected (0.028 sec)

MariaDB [rekam_medis_klinik]> CREATE TABLE OBAT (
  ->   Kode_Obat VARCHAR(10) PRIMARY KEY,
  ->   Nama_Obat VARCHAR(100) NOT NULL,
  ->   Satuan VARCHAR(20),
  ->   Harga DECIMAL(10,2)
  -> );
Query OK, 0 rows affected (0.017 sec)
```

Gambar 4. 1 SQL

```
MariaDB [rekam_medis_klinik]> CREATE TABLE KUNJUNGAN (
  ->   No_Kunjungan VARCHAR(15) PRIMARY KEY,
  ->   Tgl_Kunjungan DATE NOT NULL,
  ->   Keluhan TEXT,
  ->   Tensi VARCHAR(10),
  ->   Suhu DECIMAL(4,2),
  ->   BB DECIMAL(5,2),
  ->   No_RM VARCHAR(10),
  ->   Kode_Dokter VARCHAR(10),
  ->   FOREIGN KEY (No_RM) REFERENCES PASIEN(No_RM) ON DELETE CASCADE,
  ->   FOREIGN KEY (Kode_Dokter) REFERENCES DOKTER(Kode_Dokter) ON DELETE SET NULL
  -> );
Query OK, 0 rows affected (0.047 sec)

MariaDB [rekam_medis_klinik]> CREATE TABLE DETAIL_DIAGNOSA (
  ->   No_Kunjungan VARCHAR(15),
  ->   Kode_Diagnosa VARCHAR(10),
  ->   PRIMARY KEY (No_Kunjungan, Kode_Diagnosa),
  ->   FOREIGN KEY (No_Kunjungan) REFERENCES KUNJUNGAN(No_Kunjungan) ON DELETE CASCADE,
  ->   FOREIGN KEY (Kode_Diagnosa) REFERENCES DIAGNOSA(Kode_Diagnosa) ON DELETE CASCADE
  -> );
Query OK, 0 rows affected (0.036 sec)

MariaDB [rekam_medis_klinik]> CREATE TABLE DETAIL_RESEP (
  ->   No_Kunjungan VARCHAR(15),
  ->   Kode_Obat VARCHAR(10),
  ->   Dosis VARCHAR(50),
  ->   Jumlah INT,
  ->   PRIMARY KEY (No_Kunjungan, Kode_Obat),
  ->   FOREIGN KEY (No_Kunjungan) REFERENCES KUNJUNGAN(No_Kunjungan) ON DELETE CASCADE,
  ->   FOREIGN KEY (Kode_Obat) REFERENCES OBAT(Kode_Obat) ON DELETE CASCADE
  -> );
Query OK, 0 rows affected (0.038 sec)

MariaDB [rekam_medis_klinik]> INSERT INTO PASIEN (No_RM, Nama_Pasien, Tgl_Lahir, JK, Alamat, No_Telp) VALUES
  -> ('RM001', 'Lukman Hakim', '2005-12-04', 'L', 'Jl. Sutani, Tanjung Pinang', '0811001'),
  -> ('RM002', 'Reza Firdando', '2005-04-02', 'L', 'Jl. Engku Putri, Tanjung Pinang', '0811002'),
  -> ('RM003', 'Muhammad Fachry Reza', '2005-04-08', 'L', 'Jl. Pramuka, Tanjung Pinang', '0811003'),
  -> ('RM004', 'Maisy Bimo Ayu', '2006-04-07', 'L', 'Jl. Wiratno, Tanjung Pinang', '0811004');
Query OK, 4 rows affected (0.033 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

Gambar 4. 2 SQL

```
MariaDB [rekam_medis_klinik]> CREATE TABLE KUNJUNGAN (
  ->   No_Kunjungan VARCHAR(15) PRIMARY KEY,
  ->   Tgl_Kunjungan DATE NOT NULL,
  ->   Keluhan TEXT,
  ->   Tensi VARCHAR(10),
  ->   Suhu DECIMAL(4,2),
  ->   BB DECIMAL(5,2),
  ->   No_RM VARCHAR(10),
  ->   Kode_Dokter VARCHAR(10),
  ->   FOREIGN KEY (No_RM) REFERENCES PASIEN(No_RM) ON DELETE CASCADE,
  ->   FOREIGN KEY (Kode_Dokter) REFERENCES DOKTER(Kode_Dokter) ON DELETE SET NULL
  -> );
Query OK, 0 rows affected (0.047 sec)

MariaDB [rekam_medis_klinik]> CREATE TABLE DETAIL_DIAGNOSA (
  ->   No_Kunjungan VARCHAR(15),
  ->   Kode_Diagnosa VARCHAR(10),
  ->   PRIMARY KEY (No_Kunjungan, Kode_Diagnosa),
  ->   FOREIGN KEY (No_Kunjungan) REFERENCES KUNJUNGAN(No_Kunjungan) ON DELETE CASCADE,
  ->   FOREIGN KEY (Kode_Diagnosa) REFERENCES DIAGNOSA(Kode_Diagnosa) ON DELETE CASCADE
  -> );
Query OK, 0 rows affected (0.036 sec)

MariaDB [rekam_medis_klinik]> CREATE TABLE DETAIL_RESEP (
  ->   No_Kunjungan VARCHAR(15),
  ->   Kode_Obat VARCHAR(10),
  ->   Dosis VARCHAR(50),
  ->   Jumlah INT,
  ->   PRIMARY KEY (No_Kunjungan, Kode_Obat),
  ->   FOREIGN KEY (No_Kunjungan) REFERENCES KUNJUNGAN(No_Kunjungan) ON DELETE CASCADE,
  ->   FOREIGN KEY (Kode_Obat) REFERENCES OBAT(Kode_Obat) ON DELETE CASCADE
  -> );
Query OK, 0 rows affected (0.038 sec)

MariaDB [rekam_medis_klinik]> INSERT INTO PASIEN (No_RM, Nama_Pasien, Tgl_Lahir, JK, Alamat, No_Telp) VALUES
  -> ('RM001', 'Lukman Hakim', '2005-12-04', 'L', 'Jl. Sutani, Tanjung Pinang', '0811001'),
  -> ('RM002', 'Reza Firdando', '2005-04-02', 'L', 'Jl. Engku Putri, Tanjung Pinang', '0811002'),
  -> ('RM003', 'Muhammad Fachry Reza', '2005-04-08', 'L', 'Jl. Pramuka, Tanjung Pinang', '0811003'),
  -> ('RM004', 'Maisy Bimo Ayu', '2006-04-07', 'L', 'Jl. Wiratno, Tanjung Pinang', '0811004');
Query OK, 4 rows affected (0.033 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

Gambar 4. 3 SQL

4.3 SQL DML (Data Manipulation Language)

```
MariaDB [rekam_medis_klinik]> INSERT INTO OBAT (Kode_Obat, Nama_Obat, Satuan, Harga) VALUES
-> ('OB001', 'Paracetamol', 'Tablet', 5000), -- Untuk Demam/Pusing
-> ('OB002', 'Promag', 'Tablet', 8000), -- Untuk Maag/Lambung
-> ('OB003', 'Diapet', 'Kapsul', 6000), -- Untuk Diare
-> ('OB004', 'Woods Batuk', 'Botol', 25000); -- Untuk Batuk/ISPA
Query OK, 4 rows affected (0.011 sec)
Records: 4 Duplicates: 0 Warnings: 0

MariaDB [rekam_medis_klinik]> INSERT INTO DIAGNOSA (Kode_Diagnosa, Nama_Diagnosa) VALUES
-> ('F01', 'Febis (Demam)'),
-> ('G01', 'Gastritis (Maag)'),
-> ('D01', 'Diare Akut'),
-> ('B01', 'Batuk dan ISPA');
Query OK, 4 rows affected (0.007 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

Gambar 4. 4 SQL MDL

```
MariaDB [rekam_medis_klinik]> INSERT INTO DOKTER (Kode_Dokter, Nama_Dokter, Spesialis) VALUES
-> ('DR001', 'dr. Andi Pratama', 'Umum'),
-> ('DR002', 'dr. Heru Kusuma', 'Spesialis Penyakit Dalam'),
-> ('DR003', 'dr. Sari Wijaya', 'Spesialis Anak'),
-> ('DR004', 'dr. Budi Santoso', 'Spesialis Penyakit Dalam');
Query OK, 4 rows affected (0.014 sec)
Records: 4 Duplicates: 0 Warnings: 0

MariaDB [rekam_medis_klinik]> INSERT INTO KUNJUNGAN VALUES
-> ('K20260601', '2026-06-25', 'Demam dan sakit kepala', '120/80', 38.5, 60.0, 'RM001', 'DR001'),
-> ('K20260602', '2026-06-25', 'Myeri lambung dan mual', '110/70', 36.5, 65.0, 'RM002', 'DR001'),
-> ('K20260603', '2026-06-25', 'Buang air besar cair', '115/80', 37.0, 58.0, 'RM003', 'DR001'),
-> ('K20260604', '2026-06-25', 'Batuk berdahak', '120/80', 36.8, 62.0, 'RM004', 'DR001');
Query OK, 4 rows affected (0.009 sec)
Records: 4 Duplicates: 0 Warnings: 0

MariaDB [rekam_medis_klinik]> INSERT INTO DETAIL_DIAGNOSA (No_Kunjungan, Kode_Diagnosa) VALUES
-> ('K20260601', 'F01'), -- Lukman -> Demam
-> ('K20260602', 'G01'), -- Rezza -> Maag
-> ('K20260603', 'D01'), -- Fachry -> Diare
-> ('K20260604', 'B01'), -- Maisa -> Batuk
Query OK, 4 rows affected (0.006 sec)
Records: 4 Duplicates: 0 Warnings: 0

MariaDB [rekam_medis_klinik]> INSERT INTO DETAIL_DIAGNOSA (No_Kunjungan, Kode_Diagnosa) VALUES
-> ('K20260601', 'F01'), -- Lukman -> Demam
-> ('K20260602', 'G01'), -- Rezza -> Maag
-> ('K20260603', 'D01'), -- Fachry -> Diare
-> ('K20260604', 'B01'), -- Maisa -> Batuk
ERROR 1062 (23000): Duplicate entry 'K20260601-F01' for key 'PRIMARY'
MariaDB [rekam_medis_klinik]> INSERT INTO DETAIL_RESEP (No_Kunjungan, Kode_Obat, Dosis, Jumlah) VALUES
-> ('K20260601', 'OB001', '3x1 sesudah makan', 10),
-> ('K20260602', 'OB002', '3x1 sebelum makan', 10),
-> ('K20260603', 'OB003', '2x1 tiap mencrret', 5),
-> ('K20260604', 'OB004', '3x1 sendok teh', 1);
Query OK, 4 rows affected (0.013 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

Gambar 4. 5 SQL MDL

BAB V

PENGUJIAN DAN QUERY

5.1 Skenario Pengujian

Pengujian dilakukan dengan menjalankan berbagai query SQL terhadap basis data yang telah dibuat. Skenario pengujian dirancang untuk memvalidasi bahwa semua kebutuhan fungsional yang telah diidentifikasi dapat terpenuhi.

No	Skenario Pengujian	Target KF	Query Type
S-01	Menampilkan seluruh data pasien	KF-01	SELECT
S-02	Menampilkan riwayat lengkap satu pasien	KF-09	SELECT + JOIN
S-03	Menampilkan detail kunjungan beserta diagnosa	KF-05	SELECT + JOIN
S-04	Menampilkan detail resep obat per kunjungan	KF-06	SELECT + JOIN
S-05	Laporan statistik penyakit terbanyak	KF-10	GROUP BY + COUNT
S-06	Laporan obat paling sering diresepkan	KF-11	GROUP BY + COUNT
S-07	Menampilkan kunjungan dengan filter tanggal	KF-03	SELECT + WHERE
S-08	Menghitung total biaya resep per kunjungan	KF-06	SUM + JOIN
S-09	Update data pasien	KF-01	UPDATE
S-10	Menampilkan daftar dokter dan jumlah pasien	KF-02	GROUP BY + COUNT

5.1 QUERY SQL

Query 1 – Menampilkan Seluruh Data Pasien

```
SELECT * FROM PASIEN;
```

Query 2 – Menampilkan Nama Pasien dan Alamat Berdasarkan Lokasi

```
SELECT Nama_Pasien, Alamat  
FROM PASIEN  
WHERE Alamat LIKE '%Tanjung Pinang%';
```

Query 3 – Menampilkan Data Kunjungan Pasien (JOIN Tabel KUNJUNGAN dan PASIEN)

```
SELECT K.No_Kunjungan, P>Nama_Pasien, K.Keluhan  
FROM KUNJUNGAN K  
JOIN PASIEN P  
ON K.No_RM = P.No_RM;
```

Query 4 – Menghitung Jumlah Pasien Laki-laki

```
SELECT COUNT(*) AS Total_Pasien_Laki  
FROM PASIEN  
WHERE JK = 'L';
```

Query 5 – Laporan Riwayat Medis Kronologis (Urutan Waktu)

```
SELECT  
-> P>Nama_Pasien,  
-> K.Tgl_Kunjungan,  
-> D>Nama_Diagnosa  
-> FROM PASIEN P  
-> JOIN KUNJUNGAN K ON P.No_RM = K.No_RM  
-> JOIN DETAIL_DIAGNOSA DD ON K.No_Kunjungan = DD.No_Kunjungan  
-> JOIN DIAGNOSA D ON DD.Kode_Diagnosa = D.Kode_Diagnosa  
-> WHERE P.No_RM = 'RM001'  
-> ORDER BY K.Tgl_Kunjungan ASC;
```

Query 6 – Menampilkan Tanggal Kunjungan dan Hasil Diagnosa Pasien

```
SELECT K.Tgl_Kunjungan, D>Nama_Diagnosa  
FROM KUNJUNGAN K  
JOIN DETAIL_DIAGNOSA DD  
ON K.No_Kunjungan = DD.No_Kunjungan  
JOIN DIAGNOSA D  
ON DD.Kode_Diagnosa = D.Kode_Diagnosa  
WHERE K.No_RM = 'RM001';
```

Query 7 – Menghitung Jumlah Diagnosa Pasien

```

SELECT D>Nama_Diagnosa,
COUNT(DD.Kode_Diagnosa) AS Jumlah
FROM DETAIL_DIAGNOSA DD
JOIN DIAGNOSA D
ON DD.Kode_Diagnosa = D.Kode_Diagnosa
GROUP BY D>Nama_Diagnosa;

```

Query 8 – Menghitung Total Biaya Obat per Kunjungan

```

SELECT DR.No_Kunjungan,
SUM(O.Harga * DR.Jumlah) AS Total_Bayar
FROM DETAIL_RESEP DR
JOIN OBAT O
ON DR.Kode_Obat = O.Kode_Obat
GROUP BY DR.No_Kunjungan;

```

Query 9 – Menampilkan Data Obat dengan Harga di Atas Rp10.000

```

SELECT Nama_Obat, Harga
FROM OBAT
WHERE Harga > 10000;

```

Query 10 – Menampilkan Data Pasien Berdasarkan Nama

```

SELECT *
FROM PASIEN
WHERE Nama_Pasien = 'Rezza Firnando';

```

8.2 Hasil Pengujian

Query 1 – SELECT semua data [Tabel_Pasien]:

```

MariaDB [rekam_medis_klinik]> SELECT * FROM PASIEN;
+-----+-----+-----+-----+-----+-----+
| No_RM | Nama_Pasien | Tgl_Lahir | JK | Alamat | No_Telp |
+-----+-----+-----+-----+-----+-----+
| RM001 | Lukman Hakim | 2005-12-04 | L | Jl. Sutami, Tanjung Pinang | 0811001 |
| RM002 | Rezza Firnando | 2005-04-02 | L | Jl. Engku Putri, Tanjung Pinang | 0811002 |
| RM003 | Muhammad Fachry Reza | 2005-04-08 | L | Jl. Pramuka, Tanjung Pinang | 0811003 |
| RM004 | Maisa Bimo Ayu | 2006-04-07 | L | Jl. Wiratno, Tanjung Pinang | 0811004 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.008 sec)

```

Gambar 5. 1 QUERY 1

Query 2 – SELECT Nama Pasien dan Alamat Berdasarkan Lokasi

```

MariaDB [rekam_medis_klinik]> SELECT Nama_Pasien, Alamat FROM PASIEN WHERE Alamat LIKE '%Tanjung Pinang%';
+-----+-----+
| Nama_Pasien | Alamat |
+-----+-----+
| Lukman Hakim | Jl. Sutami, Tanjung Pinang |
| Rezza Firnando | Jl. Engku Putri, Tanjung Pinang |
| Muhammad Fachry Reza | Jl. Pramuka, Tanjung Pinang |
| Maisa Bimo Ayu | Jl. Wiratno, Tanjung Pinang |
+-----+-----+
4 rows in set (0.001 sec)

```

Gambar 5. 2 QUERY 2

Query 3 – JOIN Tabel KUNJUNGAN dan PASIEN

```
MariaDB [rekam_medis_klinik]> SELECT K.No_Kunjungan, P>Nama_Pasien, K.Keluhan FROM KUNJUNGAN K JOIN PASIEN P ON K.No_RM = P.No_RM;
```

No_Kunjungan	Nama_Pasien	Keluhan
K20260601	Lukman Hakim	Demam dan sakit kepala
K20260602	Rezza Firnando	Nyeri lambung dan mual
K20260603	Muhammad Fachry Reza	Buang air besar cair
K20260604	Maisha Bimo Ayu	Batuk berdahak

4 rows in set (0.009 sec)

Gambar 5. 3 QUERY 3

Query 4 – Menghitung Jumlah Pasien Laki-laki

```
MariaDB [rekam_medis_klinik]> SELECT COUNT(*) AS Total_Pasien_Laki FROM PASIEN WHERE JK = 'L';
```

Total_Pasien_Laki
4

1 row in set (0.007 sec)

Gambar 5. 4 QUERY 4

Query 5 – Laporan Riwayat Medis Kronologis (Urutan Waktu)

```
+-----+-----+-----+
| Nama_Pasien | Tgl_Kunjungan | Nama_Diagnosa |
+-----+-----+-----+
| Lukman Hakim | 2026-06-25    | Febris (Demam) |
+-----+-----+-----+
1 row in set (0.097 sec)
```

Gambar 5. 5 QUERY 5

Query 6 – Menampilkan Tanggal Kunjungan dan Hasil Diagnosa Pasien

```
MariaDB [rekam_medis_klinik]> SELECT K.Tgl_Kunjungan, D>Nama_Diagnosa FROM KUNJUNGAN K JOIN DETAIL_DIAGNOSA DD ON K.No_Kunjungan = DD.No_Kunjungan JOIN DIAGNOSA D ON DD.Kode_Diagnosa = D.Kode_Diagnosa WHERE K.No_RM = 'RM001';
```

Tgl_Kunjungan	Nama_Diagnosa
2026-06-25	Febris (Demam)

1 row in set (0.002 sec)

Gambar 5. 6 QUERY 6

Query 7 – Menghitung Jumlah Diagnosa Pasien

```
MariaDB [rekam_medis_klinik]> SELECT D>Nama_Diagnosa, COUNT(DD.Kode_Diagnosa) AS Jumlah FROM DETAIL_DIAGNOSA DD JOIN DIAGNOSA D ON DD.Kode_Diagnosa = D.Kode_Diagnosa GROUP BY D>Nama_Diagnosa;
```

Nama_Diagnosa	Jumlah
Batuk dan ISPA	1
Diare Akut	1
Febris (Demam)	1
Gastritis (Maag)	1

4 rows in set (0.007 sec)

Gambar 5. 7 QUERY 7

Query 8 – Menghitung Total Biaya Obat per Kunjungan

```
MariaDB [rekam_medis_klinik]> SELECT DR.No_Kunjungan, SUM(O.Harga * DR.Jumlah) AS Total_Bayar FROM DETAIL_RESEP DR JOIN OBAT O ON
DR.Kode_Obat = O.Kode_Obat GROUP BY DR.No_Kunjungan;
```

No_Kunjungan	Total_Bayar
K20260602	80000.00
K20260603	30000.00
K20260604	25000.00

3 rows in set (0.001 sec)

Gambar 5. 8 QUERY 8

Query 9 – Menampilkan Data Obat dengan Harga di Atas Rp10.000

```
MariaDB [rekam_medis_klinik]> SELECT Nama_Obat, Harga FROM OBAT WHERE Harga > 10000;
```

Nama_Obat	Harga
Woods Batuk	25000.00

1 row in set (0.002 sec)

Gambar 5. 9 QUERY 9

Query 10 – Menampilkan Data Pasien Berdasarkan Nama

```
MariaDB [rekam_medis_klinik]> SELECT * FROM PASIEN WHERE Nama_Pasien = 'Rezza Firnando';
```

No_RM	Nama_Pasien	Tgl_Lahir	JK	Alamat	No_Telp
RM002	Rezza Firnando	2005-04-02	L	Jl. Engku Putri, Tanjung Pinang	0811002

1 row in set (0.005 sec)

Gambar 5. 10 QUERY 10

BAB VI

PENUTUP

6.1 Kesimpulan

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, dapat disimpulkan beberapa hal sebagai berikut:

1. Kelompok 9 telah berhasil merancang basis data relasional untuk Sistem Rekam Medis Klinik dengan 7 tabel yang terstruktur melalui proses normalisasi hingga bentuk 3NF, yaitu: PASIEN, DOKTER, KUNJUNGAN, DIAGNOSA, OBAT, DETAIL_DIAGNOSA, dan DETAIL_RESEP.
2. Proses normalisasi berhasil menghilangkan redundansi data dan anomali yang mungkin terjadi pada pencatatan manual. Skema yang dihasilkan telah memenuhi syarat 1NF, 2NF, dan 3NF.
3. Ditemukan kebutuhan revisi logika bisnis dari Progres 1 ke Progres 2, yaitu perubahan relasi KUNJUNGAN–DIAGNOSA dari One-to-Many menjadi Many-to-Many, yang diatasi dengan penambahan entitas perantara DETAIL_DIAGNOSA.
4. Implementasi SQL DDL berhasil menciptakan struktur basis data yang lengkap dengan constraint PRIMARY KEY, FOREIGN KEY, dan tipe data yang sesuai. SQL DML berhasil mengisi data awal termasuk data dari file mdrekammedisklinik_1.md ke tabel DETAIL_DIAGNOSA.
5. Sebanyak 11 query SQL berhasil dibuat dan diuji, mencakup operasi SELECT, JOIN, GROUP BY, WHERE, UPDATE, dan DELETE, yang mampu menjawab seluruh kebutuhan fungsional sistem yang telah diidentifikasi.

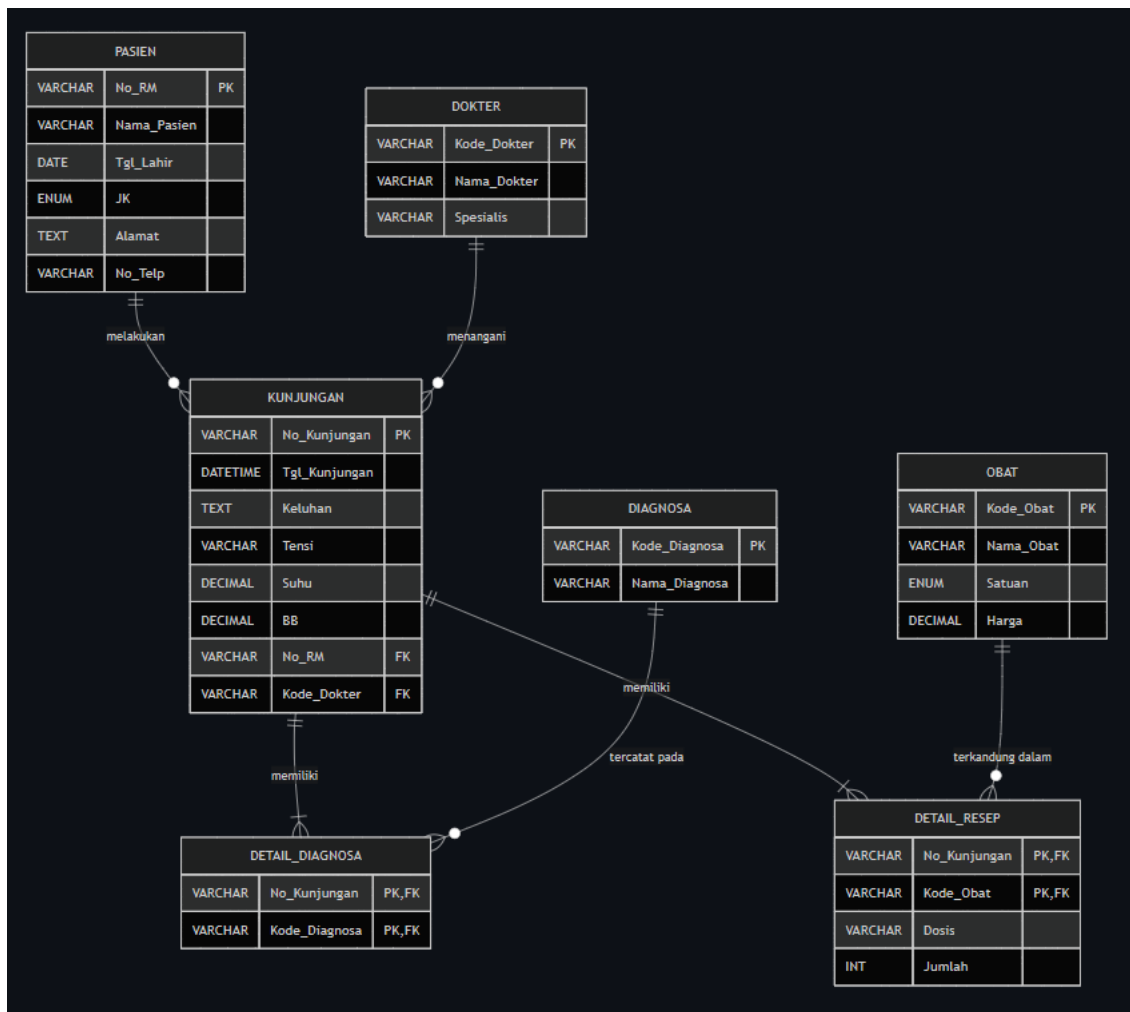
5.2 Saran

Beberapa saran untuk pengembangan sistem di masa mendatang:

1. Pengembangan antarmuka pengguna (web application atau desktop application) di atas basis data yang telah dirancang, sehingga sistem dapat langsung digunakan oleh staf klinik tanpa perlu mengetikkan perintah SQL secara langsung.
2. Penambahan fitur manajemen stok obat secara real-time, termasuk pencatatan penambahan stok (dari supplier) dan pengurangan stok otomatis setiap kali resep diproses oleh apoteker.
3. Implementasi sistem autentikasi dan otorisasi pengguna berdasarkan peran (Admin, Perawat, Dokter, Apoteker) untuk memastikan keamanan data rekam medis pasien sesuai regulasi yang berlaku.
4. Penambahan fitur backup dan restore basis data secara terjadwal untuk menjaga keamanan dan ketersediaan data rekam medis pasien dalam jangka panjang.
5. Integrasi dengan standar kode diagnosa ICD-10 secara lengkap ke dalam tabel DIAGNOSA untuk mendukung pelaporan medis yang lebih akurat dan terstandarisasi.

LAMPIRAN

Lampiran 1- ERD Full Size



Lampiran 2 - File SQL Lengkap

```
-- phpMyAdmin SQL Dump
-- version 5.2.1
-- https://www.phpmyadmin.net/
--
-- Host: 127.0.0.1
-- Waktu pembuatan: 25 Jun 2026 pada 12.01
-- Versi server: 10.4.32-MariaDB
-- Versi PHP: 8.2.12
```

```
SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
```

```
START TRANSACTION;

SET time_zone = "+00:00";


/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8mb4 */;


--
-- Database: `rekam_medis_klinik`
--

--
-- -----
--
-- Struktur dari tabel `detail_diagnosa`
--

CREATE TABLE `detail_diagnosa` (
  `No_Kunjungan` varchar(15) NOT NULL,
  `Kode_Diagnosa` varchar(10) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;


--
-- Dumping data untuk tabel `detail_diagnosa`
--

INSERT INTO `detail_diagnosa` (`No_Kunjungan`, `Kode_Diagnosa`) VALUES
('K20260601', 'F01'),
('K20260602', 'G01'),
('K20260603', 'D01'),
('K20260604', 'B01');
```



```

-----

--
-- Struktur dari tabel `detail_resep`
--

CREATE TABLE `detail_resep` (
  `No_Kunjungan` varchar(15) NOT NULL,
  `Kode_Obat` varchar(10) NOT NULL,
  `Dosis` varchar(50) DEFAULT NULL,
  `Jumlah` int(11) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Dumping data untuk tabel `detail_resep`
--

INSERT INTO `detail_resep` (`No_Kunjungan`, `Kode_Obat`, `Dosis`, `Jumlah`)
VALUES
('K20260602', 'OB002', '3x1 sebelum makan', 10),
('K20260603', 'OB003', '2x1 tiap mencret', 5),
('K20260604', 'OB004', '3x1 sendok teh', 1);

-----

--
-- Struktur dari tabel `diagnosa`
--

CREATE TABLE `diagnosa` (
  `Kode_Diagnosa` varchar(10) NOT NULL,
  `Nama_Diagnosa` varchar(150) NOT NULL

```

```

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Dumping data untuk tabel `diagnosa`
--

INSERT INTO `diagnosa` (`Kode_Diagnosa`, `Nama_Diagnosa`) VALUES
('B01', 'Batuk dan ISPA'),
('D01', 'Diare Akut'),
('F01', 'Febris (Demam)'),
('G01', 'Gastritis (Maag)');

-----

--
-- Struktur dari tabel `dokter`
--

CREATE TABLE `dokter` (
  `Kode_Dokter` varchar(10) NOT NULL,
  `Nama_Dokter` varchar(100) NOT NULL,
  `Spesialis` varchar(50) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Dumping data untuk tabel `dokter`
--

INSERT INTO `dokter` (`Kode_Dokter`, `Nama_Dokter`, `Spesialis`) VALUES
('DR001', 'dr. Andi Pratama', 'Umum'),
('DR002', 'dr. Heru Kusuma', 'Spesialis Penyakit Dalam'),
('DR003', 'dr. Sari Wijaya', 'Spesialis Anak'),
('DR004', 'dr. Budi Santoso', 'Spesialis Penyakit Dalam');

```

```
-- -----  
  
--  
-- Struktur dari tabel `kunjungan`  
--  
  
CREATE TABLE `kunjungan` (  
  `No_Kunjungan` varchar(15) NOT NULL,  
  `Tgl_Kunjungan` date NOT NULL,  
  `Keluhan` text DEFAULT NULL,  
  `Tensi` varchar(10) DEFAULT NULL,  
  `Suhu` decimal(4,2) DEFAULT NULL,  
  `BB` decimal(5,2) DEFAULT NULL,  
  `No_RM` varchar(10) DEFAULT NULL,  
  `Kode_Dokter` varchar(10) DEFAULT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;  
  
--  
-- Dumping data untuk tabel `kunjungan`  
--  
  
INSERT INTO `kunjungan` (`No_Kunjungan`, `Tgl_Kunjungan`, `Keluhan`,  
`Tensi`, `Suhu`, `BB`, `No_RM`, `Kode_Dokter`) VALUES  
( 'K20260601', '2026-06-25', 'Demam dan sakit kepala', '120/80', 38.50,  
60.00, 'RM001', 'DR001'),  
( 'K20260602', '2026-06-25', 'Nyeri lambung dan mual', '110/70', 36.50,  
65.00, 'RM002', 'DR001'),  
( 'K20260603', '2026-06-25', 'Buang air besar cair', '115/80', 37.00, 58.00,  
'RM003', 'DR001'),  
( 'K20260604', '2026-06-25', 'Batuk berdahak', '120/80', 36.80, 62.00,  
'RM004', 'DR001');  
  
-- -----
```

```
--
-- Struktur dari tabel `obat`
--

CREATE TABLE `obat` (
  `Kode_Obat` varchar(10) NOT NULL,
  `Nama_Obat` varchar(100) NOT NULL,
  `Satuan` varchar(20) DEFAULT NULL,
  `Harga` decimal(10,2) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Dumping data untuk tabel `obat`
--

INSERT INTO `obat` (`Kode_Obat`, `Nama_Obat`, `Satuan`, `Harga`) VALUES
('OB001', 'Paracetamol', 'Tablet', 5500.00),
('OB002', 'Promag', 'Tablet', 8000.00),
('OB003', 'Diapet', 'Kapsul', 6000.00),
('OB004', 'Woods Batuk', 'Botol', 25000.00);

-- -----

--
-- Struktur dari tabel `pasien`
--

CREATE TABLE `pasien` (
  `No_RM` varchar(10) NOT NULL,
  `Nama_Pasien` varchar(100) NOT NULL,
  `Tgl_Lahir` date DEFAULT NULL,
  `JK` char(1) DEFAULT NULL,
  `Alamat` text DEFAULT NULL,
```

```

`No_Telp` varchar(15) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Dumping data untuk tabel `pasien`
--

INSERT INTO `pasien` (`No_RM`, `Nama_Pasien`, `Tgl_Lahir`, `JK`, `Alamat`,
`No_Telp`) VALUES
('RM001', 'Lukman Hakim', '2005-12-04', 'L', 'Jl. Sutami, Tanjung Pinang',
'0811001'),
('RM002', 'Rezza Firnando', '2005-04-02', 'L', 'Jl. Engku Putri, Tanjung
Pinang', '0811002'),
('RM003', 'Muhammad Fachry Reza', '2005-04-08', 'L', 'Jl. Pramuka, Tanjung
Pinang', '0811003'),
('RM004', 'Maisa Bimo Ayu', '2006-04-07', 'L', 'Jl. Wiratno, Tanjung
Pinang', '0811004');

--
-- Indexes for dumped tables
--

-- Indeks untuk tabel `detail_diagnosa`
--
ALTER TABLE `detail_diagnosa`
  ADD PRIMARY KEY (`No_Kunjungan`,`Kode_Diagnosa`),
  ADD KEY `Kode_Diagnosa` (`Kode_Diagnosa`);

--
-- Indeks untuk tabel `detail_resep`
--
ALTER TABLE `detail_resep`
  ADD PRIMARY KEY (`No_Kunjungan`,`Kode_Obat`),
  ADD KEY `Kode_Obat` (`Kode_Obat`);

```

```
--  
-- Indeks untuk tabel `diagnosa`  
--  
ALTER TABLE `diagnosa`  
  ADD PRIMARY KEY (`Kode_Diagnosa`);  
  
--  
-- Indeks untuk tabel `dokter`  
--  
ALTER TABLE `dokter`  
  ADD PRIMARY KEY (`Kode_Dokter`);  
  
--  
-- Indeks untuk tabel `kunjungan`  
--  
ALTER TABLE `kunjungan`  
  ADD PRIMARY KEY (`No_Kunjungan`),  
  ADD KEY `No_RM` (`No_RM`),  
  ADD KEY `Kode_Dokter` (`Kode_Dokter`);  
  
--  
-- Indeks untuk tabel `obat`  
--  
ALTER TABLE `obat`  
  ADD PRIMARY KEY (`Kode_Obat`);  
  
--  
-- Indeks untuk tabel `pasien`  
--  
ALTER TABLE `pasien`  
  ADD PRIMARY KEY (`No_RM`);
```

```

--
-- Ketidakleluasaan untuk tabel pelimpahan (Dumped Tables)
--

--

-- Ketidakleluasaan untuk tabel `detail_diagnosa`
--

ALTER TABLE `detail_diagnosa`
  ADD CONSTRAINT `detail_diagnosa_ibfk_1` FOREIGN KEY (`No_Kunjungan`)
REFERENCES `kunjungan` (`No_Kunjungan`) ON DELETE CASCADE,
  ADD CONSTRAINT `detail_diagnosa_ibfk_2` FOREIGN KEY (`Kode_Diagnosa`)
REFERENCES `diagnosa` (`Kode_Diagnosa`) ON DELETE CASCADE;

--

-- Ketidakleluasaan untuk tabel `detail_resep`
--

ALTER TABLE `detail_resep`
  ADD CONSTRAINT `detail_resep_ibfk_1` FOREIGN KEY (`No_Kunjungan`)
REFERENCES `kunjungan` (`No_Kunjungan`) ON DELETE CASCADE,
  ADD CONSTRAINT `detail_resep_ibfk_2` FOREIGN KEY (`Kode_Obat`) REFERENCES
`obat` (`Kode_Obat`) ON DELETE CASCADE;

--

-- Ketidakleluasaan untuk tabel `kunjungan`
--

ALTER TABLE `kunjungan`
  ADD CONSTRAINT `kunjungan_ibfk_1` FOREIGN KEY (`No_RM`) REFERENCES
`pasien` (`No_RM`) ON DELETE CASCADE,
  ADD CONSTRAINT `kunjungan_ibfk_2` FOREIGN KEY (`Kode_Dokter`) REFERENCES
`dokter` (`Kode_Dokter`) ON DELETE SET NULL;
COMMIT;

/*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;
/*!40101 SET CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;
/*!40101 SET COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;

```

Lampiran 3- Dataset Uji

```
MariaDB [rekam_medis_klinik]> INSERT INTO PASIEN (No_RM, Nama_Pasien, Tgl_Lahir, JK, Alamat, No_Telp) VALUES
-> ('RM001', 'Lukman Hakim', '2005-12-04', 'L', 'Jl. Sutami, Tanjung Pinang', '0811001'),
-> ('RM002', 'Rezza Firnando', '2005-04-02', 'L', 'Jl. Engku Putri, Tanjung Pinang', '0811002'),
-> ('RM003', 'Muhammad Fachry Reza', '2005-04-08', 'L', 'Jl. Pramuka, Tanjung Pinang', '0811003'),
-> ('RM004', 'Maisa Bimo Ayu', '2006-04-07', 'L', 'Jl. Wiratno, Tanjung Pinang', '0811004');
Query OK, 4 rows affected (0.033 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
MariaDB [rekam_medis_klinik]> INSERT INTO OBAT (Kode_Obat, Nama_Obat, Satuan, Harga) VALUES
-> ('OB001', 'Paracetamol', 'Tablet', 5000), -- Untuk Demam/Pusing
-> ('OB002', 'Promag', 'Tablet', 8000), -- Untuk Maag/Lambung
-> ('OB003', 'Diapet', 'Kapsul', 6000), -- Untuk Diare
-> ('OB004', 'Woods Batuk', 'Botol', 25000); -- Untuk Batuk/ISPA
Query OK, 4 rows affected (0.011 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
MariaDB [rekam_medis_klinik]> INSERT INTO DIAGNOSA (Kode_Diagnosa, Nama_Diagnosa) VALUES
-> ('F01', 'Febris (Demam)'),
-> ('G01', 'Gastritis (Maag)'),
-> ('D01', 'Diare Akut'),
-> ('B01', 'Batuk dan ISPA');
Query OK, 4 rows affected (0.007 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
MariaDB [rekam_medis_klinik]> INSERT INTO KUNJUNGAN (No_Kunjungan, Tgl_Kunjungan, Keluhan, Tensi, Suhu, BB, No_RM, Kode_Dokter) VALUES
-> ('K20260601', '2026-06-25', 'Demam dan sakit kepala', '120/80', 38.5, 60.0, 'RM001', 'DR001'),
-> ('K20260602', '2026-06-25', 'Myeri lambung dan mual', '110/70', 36.5, 65.0, 'RM002', 'DR001'),
-> ('K20260603', '2026-06-25', 'Sering buang air besar cair', '115/80', 37.0, 58.0, 'RM003', 'DR001'),
-> ('K20260604', '2026-06-25', 'Batuk berdahak dan serak', '120/80', 36.8, 62.0, 'RM004', 'DR001');
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails ('rekam_medis_klinik`.`kunjungan`, CONSTRAINT `kunjungan_ibfk_2` FOREIGN KEY (`Kode_Dokter`) REFERENCES `dokter` (`Kode_Dokter`) ON DELETE SET NULL)
MariaDB [rekam_medis_klinik]> INSERT INTO DOKTER (Kode_Dokter, Nama_Dokter, Spesialis) VALUES
-> ('DR001', 'dr. Andi Pratama', 'Umum'),
-> ('DR002', 'dr. Heru Kusuma', 'Spesialis Penyakit Dalam'),
-> ('DR003', 'dr. Sari Wijaya', 'Spesialis Anak'),
-> ('DR004', 'dr. Budi Santoso', 'Spesialis Penyakit Dalam');
Query OK, 4 rows affected (0.014 sec)
Records: 4 Duplicates: 0 Warnings: 0

MariaDB [rekam_medis_klinik]> INSERT INTO KUNJUNGAN VALUES
-> ('K20260601', '2026-06-25', 'Demam dan sakit kepala', '120/80', 38.5, 60.0, 'RM001', 'DR001'),
-> ('K20260602', '2026-06-25', 'Myeri lambung dan mual', '110/70', 36.5, 65.0, 'RM002', 'DR001'),
-> ('K20260603', '2026-06-25', 'Buang air besar cair', '115/80', 37.0, 58.0, 'RM003', 'DR001'),
-> ('K20260604', '2026-06-25', 'Batuk berdahak', '120/80', 36.8, 62.0, 'RM004', 'DR001');
Query OK, 4 rows affected (0.009 sec)
Records: 4 Duplicates: 0 Warnings: 0

MariaDB [rekam_medis_klinik]> INSERT INTO DETAIL_DIAGNOSA (No_Kunjungan, Kode_Diagnosa) VALUES
-> ('K20260601', 'F01'), -- Lukman -> Demam
-> ('K20260602', 'G01'), -- Rezza -> Maag
-> ('K20260603', 'D01'), -- Fachry -> Diare
-> ('K20260604', 'B01'), -- Maisa -> Batuk
Query OK, 4 rows affected (0.006 sec)
Records: 4 Duplicates: 0 Warnings: 0

MariaDB [rekam_medis_klinik]> INSERT INTO DETAIL_DIAGNOSA (No_Kunjungan, Kode_Diagnosa) VALUES
-> ('K20260601', 'F01'), -- Lukman -> Demam
-> ('K20260602', 'G01'), -- Rezza -> Maag
-> ('K20260603', 'D01'), -- Fachry -> Diare
-> ('K20260604', 'B01'), -- Maisa -> Batuk
ERROR 1062 (23000): Duplicate entry 'K20260601-F01' for key 'PRIMARY'
MariaDB [rekam_medis_klinik]> INSERT INTO DETAIL_RESEP (No_Kunjungan, Kode_Obat, Dosis, Jumlah) VALUES
-> ('K20260601', 'OB001', '3x1 sesudah makan', 10),
-> ('K20260602', 'OB002', '3x1 sebelum makan', 10),
-> ('K20260603', 'OB003', '2x1 tiap mncrret', 5),
-> ('K20260604', 'OB004', '3x1 sendok teh', 1),
Query OK, 4 rows affected (0.013 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

Lampiran 4- Screenshot Hasil Query

MariaDB [rekam_medis_klinik]> SELECT * FROM PASIEN;

No_RM	Nama_Pasien	Tgl_Lahir	JK	Alamat	No_Telp
RM001	Lukman Hakim	2005-12-04	L	Jl. Sutami, Tanjung Pinang	0811001
RM002	Rezza Firnando	2005-04-02	L	Jl. Engku Putri, Tanjung Pinang	0811002
RM003	Muhammad Fachry Reza	2005-04-08	L	Jl. Pramuka, Tanjung Pinang	0811003
RM004	Maisa Bimo Ayu	2006-04-07	L	Jl. Wiratno, Tanjung Pinang	0811004

4 rows in set (0.008 sec)

MariaDB [rekam_medis_klinik]> SELECT Nama_Pasien, Alamat FROM PASIEN WHERE Alamat LIKE '%Tanjung Pinang%';

Nama_Pasien	Alamat
Lukman Hakim	Jl. Sutami, Tanjung Pinang
Rezza Firnando	Jl. Engku Putri, Tanjung Pinang
Muhammad Fachry Reza	Jl. Pramuka, Tanjung Pinang
Maisa Bimo Ayu	Jl. Wiratno, Tanjung Pinang

4 rows in set (0.001 sec)

MariaDB [rekam_medis_klinik]> SELECT K.No_Kunjungan, P>Nama_Pasien, K.Keluhan FROM KUNJUNGAN K JOIN PASIEN P ON K.No_RM = P.No_RM;

No_Kunjungan	Nama_Pasien	Keluhan
K20260601	Lukman Hakim	Demam dan sakit kepala
K20260602	Rezza Firnando	Nyeri lambung dan mual
K20260603	Muhammad Fachry Reza	Buang air besar cair
K20260604	Maisa Bimo Ayu	Batuk berdahak

4 rows in set (0.009 sec)

MariaDB [rekam_medis_klinik]> SELECT K.Tgl_Kunjungan, D>Nama_Diagnosa FROM KUNJUNGAN K JOIN DETAIL_D IAGNOSA DD ON K.No_Kunjungan = DD.No_Kunjungan JOIN DIAGNOSA D ON DD.Kode_Diagnosa = D.Kode_Diagnosa WHERE K.No_RM = 'RM001';

Tgl_Kunjungan	Nama_Diagnosa
2026-06-25	Febris (Demam)

1 row in set (0.002 sec)

MariaDB [rekam_medis_klinik]> SELECT COUNT(*) AS Total_Pasien_Laki FROM PASIEN WHERE JK = 'L';

Total_Pasien_Laki
4

1 row in set (0.007 sec)

```
MariaDB [rekam_medis_klinik]> SELECT D>Nama_Diagnosa, COUNT(DD.Kode_Diagnosa) AS Jumlah FROM DETAIL_
DIAGNOSA DD JOIN DIAGNOSA D ON DD.Kode_Diagnosa = D.Kode_Diagnosa GROUP BY D>Nama_Diagnosa;
```

Nama_Diagnosa	Jumlah
Batuk dan ISPA	1
Diare Akut	1
Febris (Demam)	1
Gastritis (Maag)	1

4 rows in set (0.007 sec)

```
MariaDB [rekam_medis_klinik]> SELECT DR.No_Kunjungan, SUM(O.Harga * DR.Jumlah) AS Total_Bayar FROM D
ETAIL_RESEP DR JOIN OBAT O ON DR.Kode_Obat = O.Kode_Obat GROUP BY DR.No_Kunjungan;
```

No_Kunjungan	Total_Bayar
K20260602	80000.00
K20260603	30000.00
K20260604	25000.00

3 rows in set (0.001 sec)

```
MariaDB [rekam_medis_klinik]> SELECT Nama_Obat, Harga FROM OBAT WHERE Harga > 10000;
```

Nama_Obat	Harga
Woods Batuk	25000.00

1 row in set (0.002 sec)

MariaDB server version for the right syntax to use near 'mysqldump' at line 1

```
MariaDB [rekam_medis_klinik]> SELECT * FROM OBAT WHERE Kode_Obat = 'OB001';
```

Kode_Obat	Nama_Obat	Satuan	Harga
OB001	Paracetamol	Tablet	5500.00

1 row in set (0.020 sec)

```
MariaDB [rekam_medis_klinik]> SELECT * FROM PASIEN WHERE Nama_Pasien = 'Rezza Firnando';
```

No_RM	Nama_Pasien	Tgl_Lahir	JK	Alamat	No_Telp
RM002	Rezza Firnando	2005-04-02	L	Jl. Engku Putri, Tanjung Pinang	0811002

1 row in set (0.005 sec)

```

+-----+-----+
| No_Kunjungan | Kode_Diagnosa |
+-----+-----+
| K20260602    | G01           |
+-----+-----+
1 row in set (0.208 sec)

```

```

+-----+-----+-----+
| Nama_Pasien | Tgl_Kunjungan | Nama_Diagnosa |
+-----+-----+-----+
| Lukman Hakim | 2026-06-25    | Febris (Demam) |
+-----+-----+-----+
1 row in set (0.097 sec)

```

Lampiran 5- Link Repository GITHUB

<https://github.com/rfahr331/PBL-SistemBasisData-RekamMedisKlinik-Kel9>